



# TH-GUITARS: Documentazione Tecnica del Progetto

## 1. Panoramica dell'Architettura

TH-Guitars è un'applicazione web E-commerce full-stack basata sul pattern **Client-Server**.

- **Frontend:** HTML5, CSS3 (con framework Bootstrap 5 per la responsività) e JavaScript Vanilla (ES6+).
- **Backend:** PHP (con PDO per la sicurezza contro le SQL Injection).
- **Database:** MySQL.
- **Comunicazione:** Le pagine HTML non vengono mai ricaricate dal server tramite POST tradizionali; il frontend usa le API `fetch` in JavaScript per dialogare in modo asincrono con il backend scambiando dati esclusivamente in formato **JSON**.

---

## 2. Struttura del Database (MySQL)

Il database `thomann` è normalizzato e diviso in 5 tabelle relazionali:

- **utenti:** Gestisce l'anagrafica. Salva `email` (univoca), `password` (criptata tramite `password_hash` in PHP) e il `ruolo` (che può essere `user` o `admin`).
- **categorie:** Tabella di appoggio per le tipologie di strumenti (Elettriche, Acustiche, Bassi).
- **prodotti:** Il catalogo. Contiene `nome`, `prezzo`, `giacenza` (stock di magazzino), il riferimento alla `categoria` (Foreign Key) e il percorso dell'immagine `url`.
- **carrello:** Tabella ponte. Associa un `id_utente` a un `id_prodotto`, registrandone la quantità. Viene svuotata al momento del checkout.
- **ordini:** Salva lo storico acquisti. Memorizza l'`id_utente`, il `totale_euro`, la `data_ordine` e lo `stato` dell'ordine (in attesa, confermato, spedito, consegnato).

---

## 3. Backend: Endpoint API (PHP)

La cartella `api/` funge da vero e proprio motore RESTful. Ogni file riceve richieste (GET, POST, DELETE) e restituisce un JSON.

### Gestione Utenti e Autenticazione

- **db.php:** File di configurazione. Crea l'oggetto PDO per la connessione al database.
- **register.php:** Riceve i dati del form di registrazione. Cripta la password e salva il nuovo utente imponendo di default il ruolo `user`.
- **login.php:** Cerca l'utente tramite email, verifica la password criptata con `password_verify` e, se corretta, salva in `$_SESSION` l'ID e il ruolo dell'utente.

- **me.php**: Endpoint fondamentale. Verifica se esiste una sessione attiva e restituisce al frontend i dati dell'utente loggato (incluso il ruolo) per disegnare la UI corretta (es. bottone Admin).
- **logout.php**: Distrugge la sessione PHP e chiude l'accesso.
- **profilo.php**: Permette all'utente di recuperare (GET) o aggiornare (POST) il proprio indirizzo di spedizione e cambiare la password.

## Catalogo e Carrello

- **prodotti.php**: Estrae i prodotti dal DB. Supporta tre filtri passati via query string: `categoria`, `search` (ricerca testuale) e `sort` (ordinamento per prezzo o nome).
- **prodotto.php**: Estrae i dettagli di un singolo prodotto tramite il suo ID.
- **carrello.php**: Gestisce il CRUD del carrello dell'utente loggato.
  - **GET**: Restituisce i prodotti nel carrello.
  - **POST**: Aggiunge un prodotto o ne incrementa la quantità.
  - **DELETE**: Rimuove un prodotto dal carrello.
- **checkout.php**: Elabora l'ordine. Calcola il totale dal carrello, riduce la `giacenza` dei prodotti nel DB, crea un record in `ordini` e svuota il carrello.

## Pannello Amministratore (Admin)

Tutti questi file hanno un controllo iniziale che blocca l'esecuzione se `$_SESSION['ruolo']` non è 'admin'.

- **admin\_stats.php**: Restituisce i dati per la dashboard: numero totale di ordini, incasso totale e un array di prodotti in esaurimento (`giacenza <= 3`).
- **admin\_ordini.php**: Permette (GET) di vedere tutti gli ordini del negozio e (POST) di cambiarne lo stato di spedizione.
- **admin\_prodotti.php**: Permette all'admin di inserire nuovi prodotti o eliminarli (DELETE) dal catalogo.

---

## 4. Frontend: Logica JavaScript (app.js)

Il file `frontend/javascript/app.js` è il cervello del client. Usa funzioni asincrone (`async/await`).

### Moduli Principali:

1. **Motore di Sessione (`controllaSessione`)**: Gira all'avvio di ogni pagina. Chiama `api/me.php`. Se l'utente è loggato, inietta nella Navbar il suo nome, il carrello e, se è Admin, il bottone per il pannello di controllo. Se la pagina è protetta (es. Ordini) e l'utente non è loggato, lo reindirizza automaticamente al login.
2. **Gestione Catalogo e Ricerca**: `caricaCatalogo()` legge i valori dalla barra di ricerca e dalla tendina dei filtri, costruisce l'URL (es. `api/prodotti.php?search=Fender&sort=prezzo_asc`) e disegna le card dei prodotti nell'HTML. È supportato anche un sistema di *Autocomplete* AJAX che suggerisce i prodotti mentre si digita.

3. **Carrello Dinamico:** Il badge del carrello è gestito da `aggiornaContatoreCarrello()`. La pagina carrello disegna gli elementi calcolando il subtotale. Al checkout, `eseguiCheckout()` invia l'indirizzo e finalizza l'ordine bloccando i bottoni per evitare doppi click.
4. **Pannello Admin:** Funzioni dedicate come `caricaTuttiOrdini()`, `caricaAdminProdotti()` e `aggiornaStato()` permettono di manipolare il database direttamente dalle interfacce protette, passandosi pacchetti JSON in POST.

---

## 5. Flussi Utente (User Journey)

- **Il flusso di Acquisto:**
  1. L'utente atterra su `index.html` e filtra i prodotti.
  2. Clicca "Aggiungi al carrello". JS intercetta l'ID, controlla il login e invia l'ID a `carrello.php` in POST.
  3. Il server controlla se il prodotto è già nel carrello: se sì fa `UPDATE quantita = quantita + 1`, altrimenti fa `INSERT`.
  4. L'utente va nel carrello, compila l'indirizzo e preme Checkout.
  5. `checkout.php` scala le scorte, genera l'ordine, svuota il carrello e rimanda alla pagina "I miei ordini".
- **Il flusso Admin:**
  1. L'admin effettua il login. Il DB riconosce il ruolo `admin` e lo passa al JS tramite `me.php`.
  2. Il JS inietta il bottone rosso "⚙️ ADMIN" nella navbar.
  3. Entrando nel pannello, l'admin visualizza le statistiche in tempo reale e ha pieno controllo sugli stati degli ordini dei clienti e sulle giacenze del magazzino.

---

## 6. Sicurezza e Ottimizzazioni

- **Protezione Password:** Le password non sono mai salvate in chiaro, ma trattate con algoritmi di hashing asimmetrico (`password_hash`).
- **Prevenzione SQL Injection:** Tutte le query al database utilizzano i **Prepared Statements** (`$stmt->prepare()` e `execute([$valore])`), separando nettamente i comandi SQL dai dati inseriti dagli utenti.
- **Protezione Rotte:** Le API verificano sempre il cookie di sessione. Anche se un utente normale provasse a chiamare `admin_ordini.php` forzando l'URL o Postman, il server risponderebbe con `HTTP 403 Forbidden` rifiutando l'esecuzione.